LIGHTNING TALK · 10 MINUTES

Harness Engineering

Every agent is a model inside the same five primitives. Here they are, and how to make them production-ready.

Chase Dovey · Average Joes Lab

github.com/averagejoeslab/harness-engineering

```
agent = harness(model)
```

THE ONE IDEA

A model only predicts the next token. The harness is everything that turns that into an agent.

Agent = Harness(Model)

THE AGENT LOOP

Observe → think → act, and repeat

Observe	Think	Act
the dark, a floor, silence	where am I? find out	reach out
something flat and cool	a wall, so a room: find a switch	slide your hand along it
smooth, smooth, then a small edge	that's a switch	flip it

Light. Now you can see where you are, and the loop repeats with whatever you notice next. Every turn started from what the last one taught you.

Agent = Harness(Model)

The model: the mind

Does the thinking. Tokens go in, and it predicts the next one.

The harness: the body

Captures input, assembles context, calls the model, handles its output, and defines the loop.

A model alone is a brain in a jar. A harness alone is a body without a head.

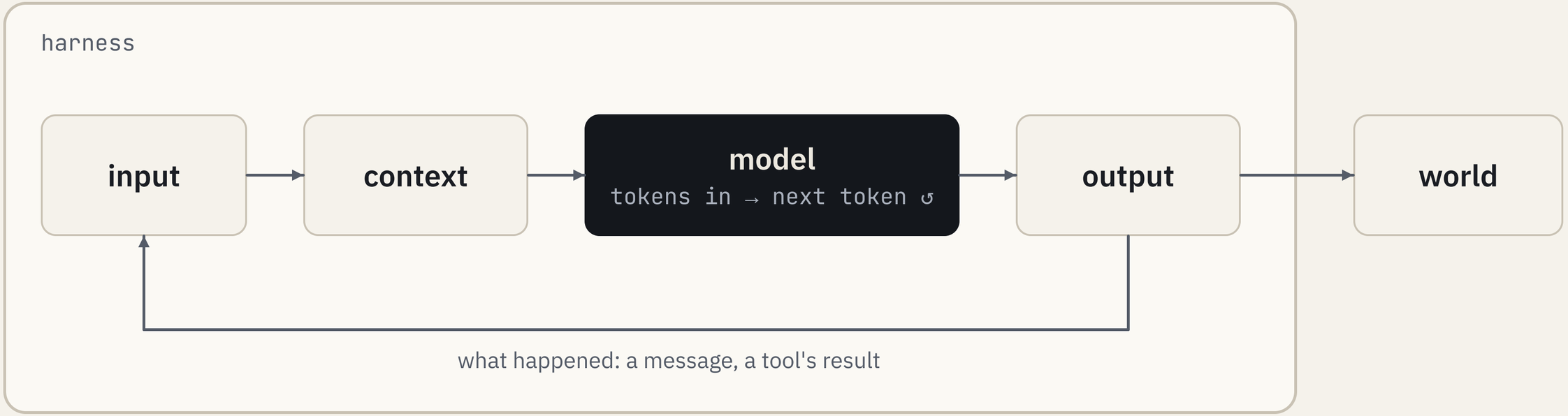
THE HARNESS

The five harness primitives

Primitive	What it does	For example
Input	captures what comes in	a message, the result of an action
Context	assembles what the model is shown	instructions, the message
Model interface	requests a response from the model	an API, a local model
Output	handles the model's outputs	a reply, an action
Control flow	defines the shape of the loop	go again, hand back, stop

THE HARNESS

The model's loop, inside the harness



Input captures, context assembles, the **model interface** makes the call, output acts on the world, and **control flow** is the shape of the whole loop.

STEP 4 · THE RUN

It finds the bug, fixes it, and checks

```
uv run ../4_control_flow.py
```

```
< The tests are failing. Find out why and fix it.
$ ls; git status --short | head; (npm test || pytest || make test || cargo test || go test ./...) 2>&1 | tail -50
  └ …  test_prices.py .F  …  E       AssertionError: 15 == 22.5
$ cat prices.py test_prices.py
  └ def total(items, discount=0):
        return round(subtotal - discount, 2)  …
> The docstring says the discount is a percentage, but the code subtracts it as a flat amount.
$ sed -i 's/return round(subtotal - discount, 2)/return round(subtotal * (1 - discount \/ 100), 2)/' …
$ pytest -q 2>&1 | tail -3; cat prices.py
  └ 2 passed in 0.01s
> The tests pass now: both of them, when I re-ran `pytest`.
```

This is an agent: it keeps going until the task is finished, then hands back to you.

PRODUCTION CONCERNS

Six questions of every primitive

Question	Concern
What should outlast the session?	Persistence
What could do harm, and who could make it?	Safety & Security
What did it actually do?	Observability
What if something breaks?	Resilience
Why is it slow or costly?	Performance
How do we know it still works?	Evaluation

Each answer adds a few lines to harness.py, marked with its question. The result is harness_prod.py, which runs commands in Docker.

SAFETY & SECURITY

What could do harm, and who could make it?

Safety

Guardrails against the agent's **own mistakes**: a box that sees only this folder, no network, and nothing runs without your yes.

Security

Guardrails against **someone turning it against you**: text it reads, secrets in its folder, a poisoned memory, a tampered image.

Both are guardrails, for different purposes. The guards that matter sit in the harness, where no text can talk them out of their job.

RECAP

What each primitive gave the agent

Primitive	What it gave the agent
Model interface	a way to reach the model
Input	a way to be asked
Output	a way to act
Control flow	a way to keep going until the task is done
Context	knowing what it is and how to use its body, and room to keep working

Then you evaluated it on tasks, and got it ready for production by asking six questions of every primitive. **That's harness engineering.**

FROM THE PAPER

Every agent is these five

	Claude Code	Codex CLI	opencode	pi	Hermes Agent	LangGraph
Input	prompts, stdin, queued messages	op channel, steer queue, mailbox	durable inbox: steer or queue	TUI, print, RPC; steer, follow-up	one event from CLI, 25 gateways	writes to a typed state
Context	cached prefix, CLAUDE.md, compaction	Responses items, world-state diffs	event-sourced history, compaction	session tree, sectioned prompt	3-tier prompt, 50% compaction	state messages; fitting is user code
Model interface	Messages API, 10 retries, fallback	Responses API, WS then HTTP	in-house request per protocol	one stream fn, 15 adapters	pluggable transports, fallback chain	one LangChain call per step
Output	built-in + MCP, OS sandbox	approve, sandbox, run; V8 code mode	concurrent tools, code-mode interpreter	parallel tools, QuickJS code mode	sandboxed terminal, approval gate	tool node, concurrent in-process
Control flow	loop till no tool call; permission gate; depth 3	task per turn; Guardian; sub-sessions	fibre per session; child sessions	inner, outer, session loops	phase-verdict loop; stop-gate nudges	a user-written graph

Six harnesses, read at pinned commits. Every component had a home in one primitive.

KEEP GOING

Where to go next

`github.com/averagejoeslab/harness-engineering`	Every step and every run in this course, reproducible from shop/.
`github.com/averagejoeslab/building-agents`	The book and the paper: building and training the model too.
`Contribute`	The book is a draft. Reviewers and contributors are welcome: open an issue or a pull request.

Questions?